Libra Library OS

PRODUCT OVERVIEW · OCTOBER 2026

A calmer way to run a *library*.

Catalog, circulation, members, payments, events, spaces, personal shelves, analytics and audit — one coherent, working system for the whole library.

FastAPI | SQLite / PostgreSQL | Responsive PWA | Reading goals | Audit-first

01

Run the desk

Circulation, members, fines and finance in one ledger.

02

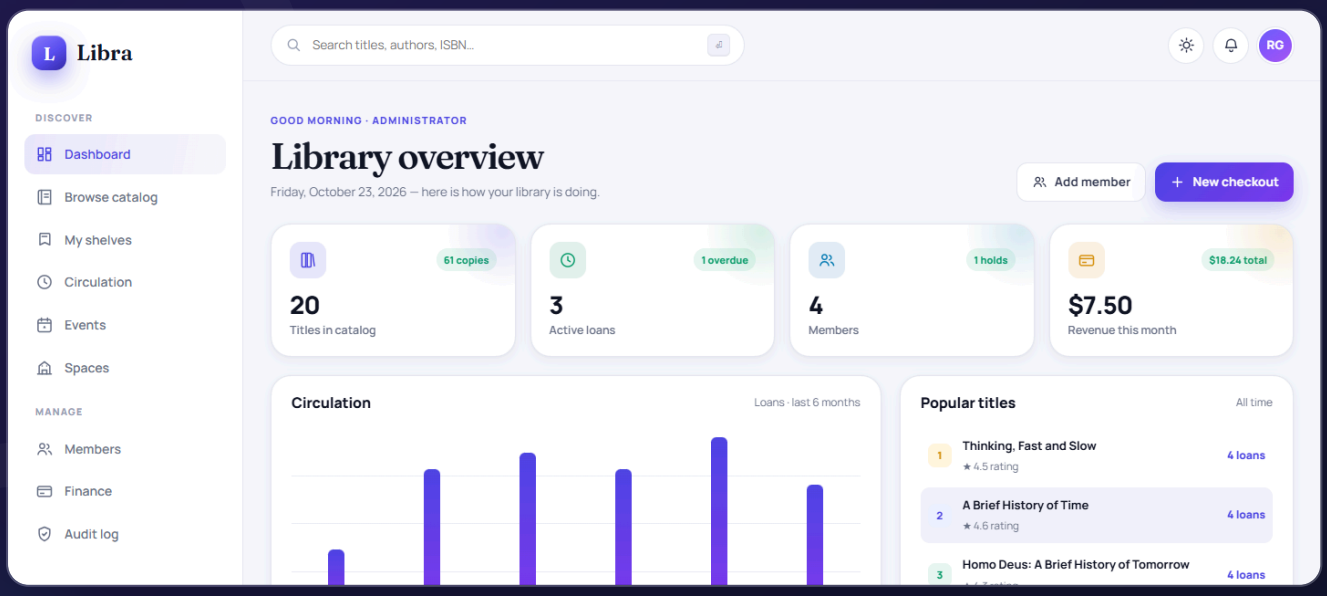
Welcome readers

Discovery, shelves, reading goals, holds, events and rooms.

03

Know the story

Usage trends, community activity and a searchable audit trail.



LIVE DEMO · LOCALHOST:8000 | 20 TITLES · 61 COPIES · 4 BOOKABLE SPACES | 20 DOCUMENTED SCREENS

Library work, joined up.

Most small libraries still stitch together spreadsheets, circulation counters, booking forms and payment links. Libra gives staff one calm workspace and gives readers one clear place to discover, borrow, pay and belong.

20
seeded titles · 61 physical copies · 2 branches

77
documented API operations across 12 route modules

3
role-based experiences: admin, librarian and member

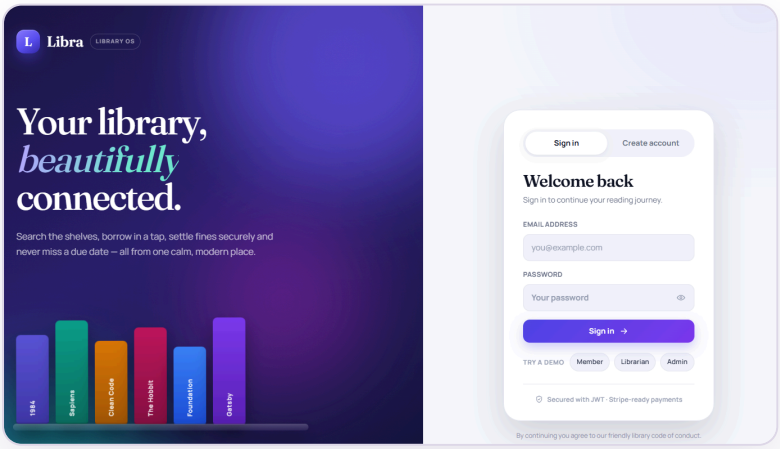
20
reviewed desktop, dark-mode and mobile screens

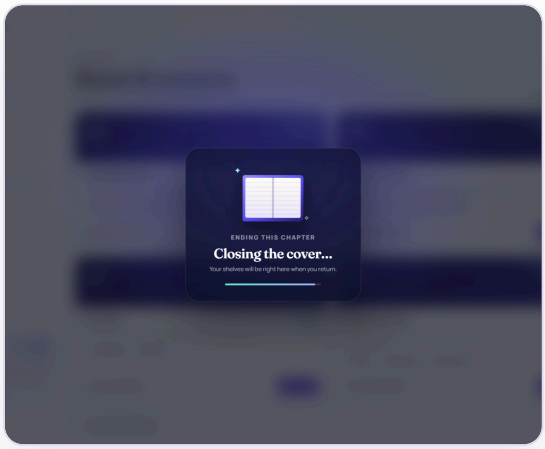
What ships, end to end

- **Catalog & inventory** — search, filters, ISBNs, authors, publishers, categories, branch copies and availability.
- **Circulation** — checkout, renew, return, holds, overdue detection, daily fines and staff desk workflows.
- **Members & access** — library cards, tiers, roles, JWT sessions, password change and staff controls.
- **Money** — fine payment, memberships, event tickets, payment history and finance analytics.
- **Reading life** — private want/reading/finished shelves, progress and an annual goal that counts finished books.
- **Community** — events, registrations, bookable rooms and conflict-safe space booking.
- **Insight** — circulation and revenue trends, popular titles, member reading history and recommendations.
- **Trust** — transactional audit ledger, permission guards and a clear view of who changed what.
- **Delivery** — installable PWA, offline app shell, light/dark themes and a mobile bottom navigation.

THE CORE BET

A library system should feel like a good library: calm, legible, trustworthy, and built around the people who use it.





First and last impression. A staggered split-screen sign-in with one-click demo roles and password reveal (left); a book-cover transition enters the app, and logout closes the cover with a quiet reading promise (right).

Built around how libraries actually work.

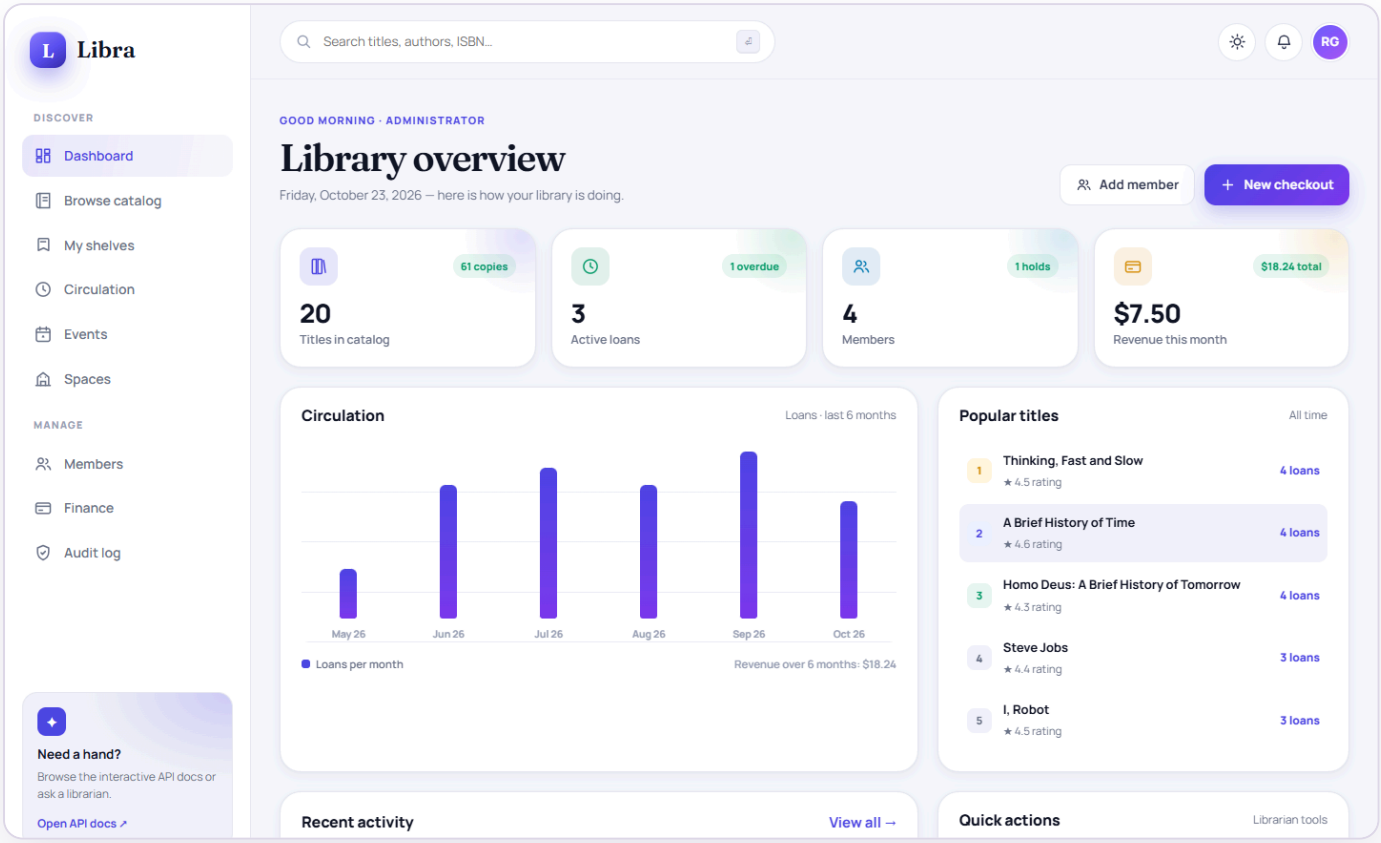
The product direction comes from public-library practice, open-source ILS conventions, accessibility standards and current payments infrastructure. Each research signal has a concrete response in the build.

SIGNAL	WHAT THE SECTOR SAYS	LIBRA'S RESPONSE
Libraries are community infrastructure	IFLA and ALA frame libraries as trusted spaces for learning, connection and access, not just book storage. [1][2][3]	Programs and events, four free bookable rooms, digital editions, in-app notices and a member-first dashboard.
Circulation is the trust contract	Open-source ILS practice centres on patrons, items, loans, holds and fines with clear state transitions. [6]	Branch-level copies, availability accounting, renewals, hold expiry, transparent due dates, daily fines and audited desk actions.
Decisions need evidence	National library statistics and academic-library reporting emphasise usage, outcomes and operational visibility. [4][5]	Six-month circulation and revenue trends, popular titles, overdue/hold indicators, member reading stats and a searchable audit ledger.
Access happens on a phone	Public-library reporting and the Seattle Public Library app launch show that patrons expect account tools and discovery on the move; accessibility is a baseline. [17][18][15][16]	Mobile-first SPA with bottom navigation, personal shelves, goals, keyboard handling, focus states, reduced-motion support and an installable PWA shell.
Readers want a personal shelf	Libby, Literal and Aspen Discovery all make saved, in-progress and finished reading visible, with progress and yearly reading views. [19][20][21]	Private want/reading/finished shelves, one-tap saving, a progress slider and an annual goal calculated from finished books.
Motion must earn its keep	NN/g recommends roughly 100–500 ms for most UI animation and a reduced-motion alternative. [22]	Short, purposeful login/logout transitions; a reduced-motion mode removes the theatre while preserving every state change.
Payments must be safe by default	Stripe's hosted Checkout keeps card data off the application server and supports wallets and local payment methods. [11][12]	Hosted Checkout for fines, memberships and tickets; signed webhooks; return-page confirmation; a clearly labelled development mode when keys are absent.
Metadata is moving to linked data	BIBFRAME provides a path from record-centric catalogs toward linked-data description. [7]	Structured ISBN, author, publisher and category fields today; a defined seam for BIBFRAME/ MARC export tomorrow.

> "The best library software disappears into the work — and leaves a clear record behind."
>
> Design principle used throughout Libra

The command centre.

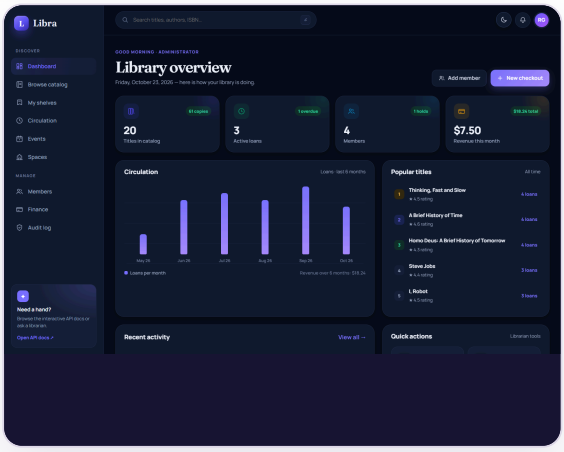
The dashboard answers the questions staff ask first: what is out, what is late, what is popular, and what needs attention now.



Dashboard, 1440 px. Four live stat cards, circulation and revenue trends, popular titles, recent activity and role-aware quick actions.

Designed for motion, not noise

- Counters and bars animate once; the interface settles quickly.
- Overdue, holds and revenue use text as well as colour.
- Every quick action opens the task, not a dead-end summary.



Dark theme is remembered per device and keeps the same information hierarchy.

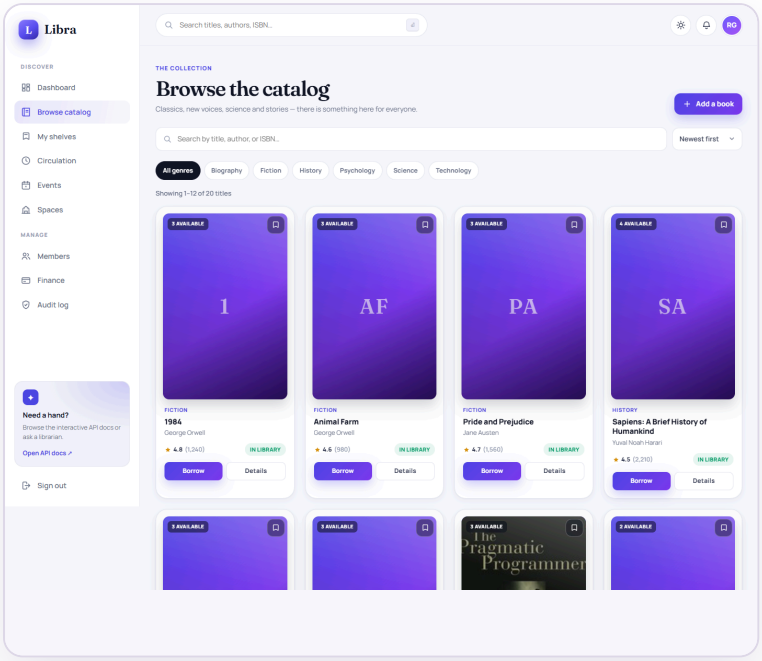
3
overdue loans surfaced on the member and staff dashboards

1
hold waiting for pickup in the seeded circulation queue

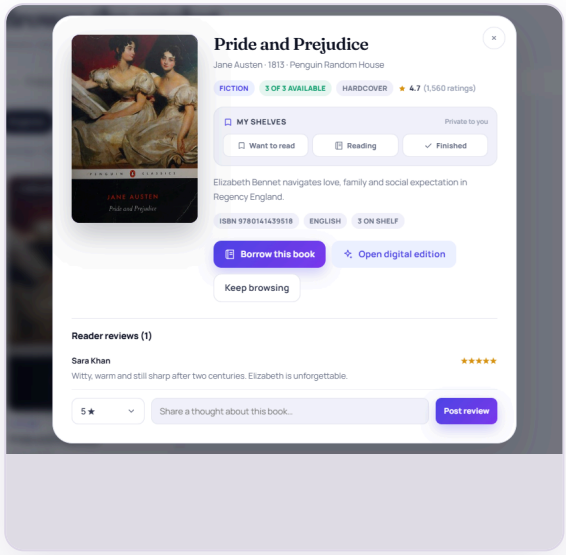
$7.50
revenue this month, with every transaction in the ledger

Live seed values at the time of capture. Staff can move directly from each signal to the underlying record.

A catalog that reads like a shelf.



Browse. Search by title, author or ISBN; filter by category and format; sort by newest, title, rating or popularity.



Book detail. Cover, publisher, year, ISBN, availability, ratings, reviews — plus one-tap shelf controls and reading progress.

Inventory

Every title can hold multiple physical copies, each with a barcode, branch, condition, shelf location and state.

Digital

Public-domain editions link to Project Gutenberg; covers are sourced from Open Library and degrade gracefully to typographic placeholders.

Intelligence

A transparent local keyword heuristic suggests category, tags and a starter description for staff — always editable, never a black box.

Data honesty: aggregate catalog ratings are labelled *ratings*; written member reviews are a separate list. A new review folds into the running rating average without erasing the imported count.

20
titles across fiction, history, science and more

61
physical copies with branch, shelf and condition

17
authors and 6 categories in the demo catalog

5
written reviews, separate from imported rating counts

TRY IT

Search by title, author or ISBN; save a card to “Want to read” with the bookmark pin; open it to set progress and manage every shelf.

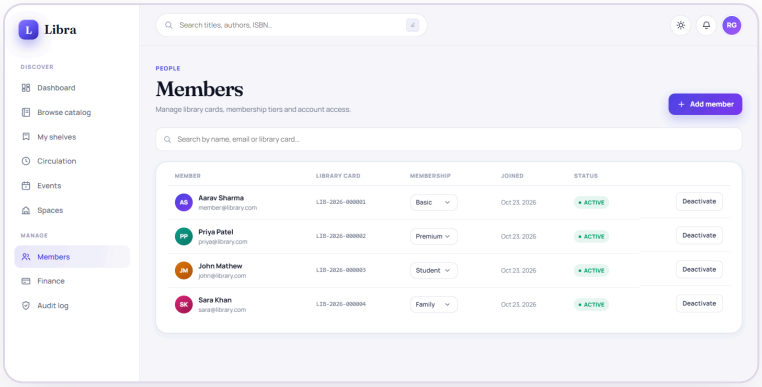
sapiens · octavia butler · 9780141439518

WHAT STAFF CAN FIX

Add a title, attach copies, edit metadata, mark condition, move a copy between branches and correct a category — without leaving the catalog.

The work behind the desk.

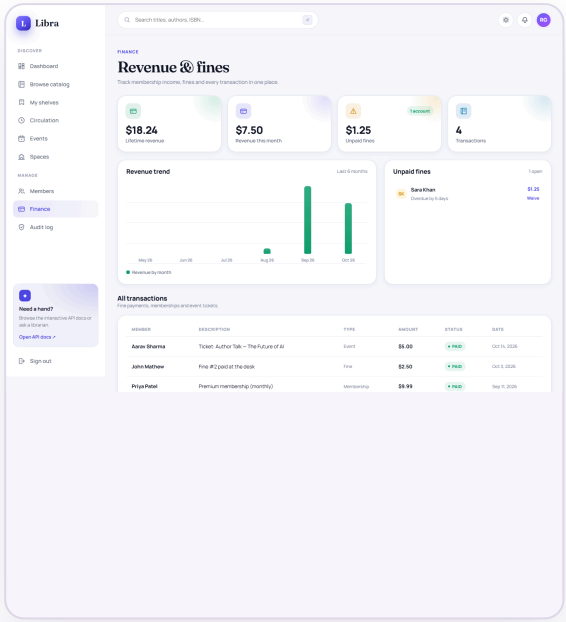
Loans, holds, member records and money share one source of truth. Staff can act quickly; members can self-serve; every sensitive change leaves a trail.



Members. Library cards, tiers, role and account status, with search by name, email or card number.

Circulation states are explicit

Active → renewed → returned, with overdue detection and fine calculation on return. Holds move through pending, ready and fulfilled with a pickup window.



Finance. Lifetime and monthly revenue, outstanding fines, transaction history and a live revenue trend.

Fine policy

Configurable daily rate, automatic calculation, desk settlement, member checkout and staff waiver.

Membership tiers

Basic, Student, Premium and Family plans with monthly pricing and 30-day periods.

Transactional audit

The audit row commits in the same transaction as the action it describes, so the record cannot drift.

01 · Check out

Scan or search the card, then the copy. Availability is checked before the loan is created.

02 · Set the date

Due date, branch and copy state move together; the member sees the same date immediately.

03 · Renew or return

Renewals extend the loan; returns calculate a fine from the configured daily rate.

04 · Settle or waive

Payment or a staff waiver closes the fine, writes the ledger row and updates the member record.

One loop, four states. The desk workflow is mirrored in the member UI, so staff and readers see the same truth from different angles.

Loan → fine

A late return creates a fine against the member and the loan, with the daily rate recorded.

Payment → receipt

Settlement stores the payment reference, method and amount beside the fine it closes.

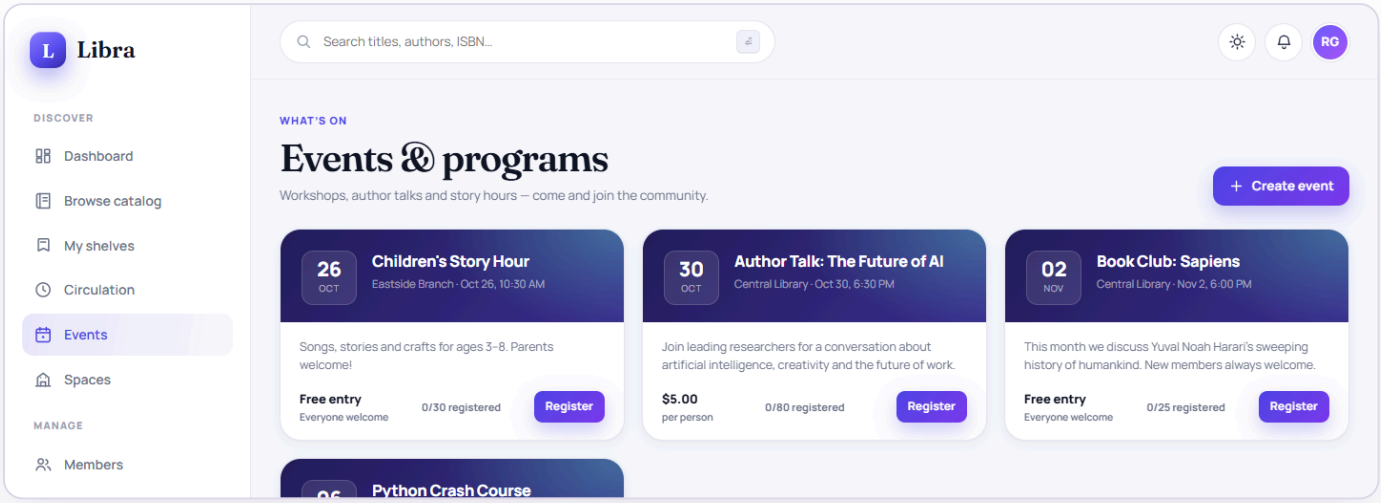
Membership → period

A paid tier activates a dated membership period and appears in the member's account.

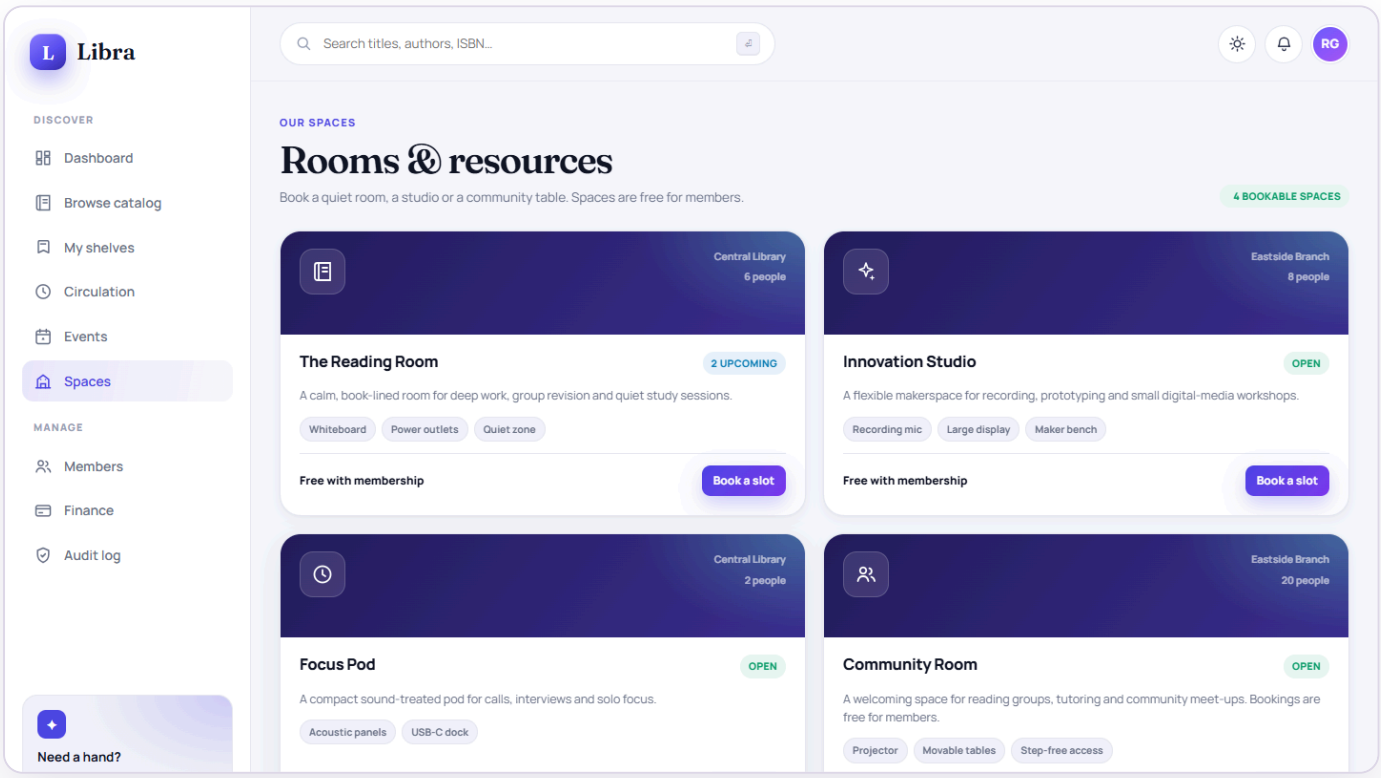
> "The desk and the member should never disagree about what is owed."
>
> One ledger, two views

Programs, rooms and reasons to gather.

Events and spaces turn the catalog into a community calendar. Booking is free for members, capped at four hours, and protected against overlaps.



Events. Story hours, author talks, book clubs and workshops with capacity, pricing, branch and registration counts.



Spaces. Four bookable resources — reading room, innovation studio, focus pod and community room — with amenities, capacity and upcoming bookings.

CONFLICT-SAFE BY DESIGN

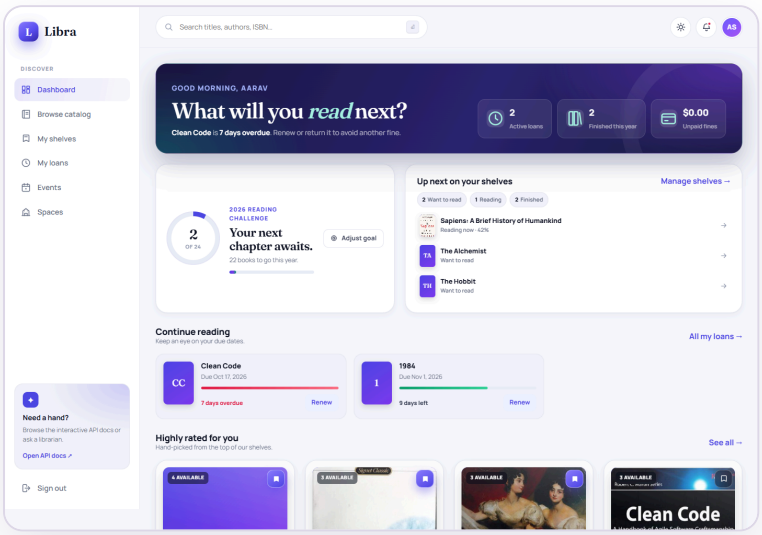
The booking endpoint rejects overlaps, enforces the four-hour member limit and keeps cancellation in the member's hands.

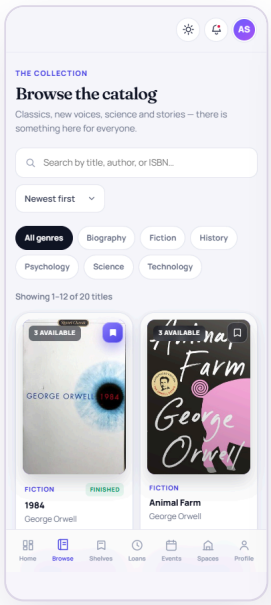
USEFUL FOR STAFF, TOO

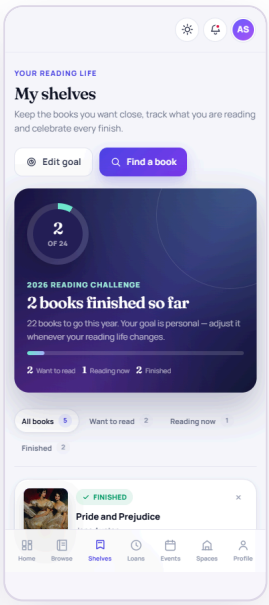
Staff see all upcoming bookings across rooms and can book on behalf of a member from the same screen.

What will you read next?

The member home leads with the next action, not a wall of data: what is due, what is late, what is recommended, what is waiting on a shelf, and how the year is going.







Member dashboard. Overdue copy in plain language, loan progress, a live goal ring, shelf counts, "Up next" and recommendations grounded in ratings and availability.

Mobile browse and shelves at 390 px. Bottom navigation replaces the desktop sidebar; no horizontal overflow; filters, shelf tabs and the goal ring remain usable with one hand.

Self-service

Members can borrow, renew, return, reserve, pay fines, book rooms, register for events, curate shelves, set a goal and manage their profile.

Clear language

"7 days overdue" and "9 days left" replace ambiguous dates; empty states explain what to do next.

Quietly inclusive

Keyboard escape, visible focus, reduced motion, semantic headings and colour-plus-text status labels.

Borrow

See availability, place a hold or check out from the catalog.

Renew

Extend a loan in place, with the new due date made explicit.

Return

Close the loop; any fine lands in the member's own ledger.

Review

Leave a rating and written review; recommendations improve.

What is due?

The next return is the first thing a member sees, with plain-language lateness and a direct renew action.

What is next?

Current loans, reserved titles and the "Up next" shelf preview continue the reading thread; the goal ring shows the year at a glance.

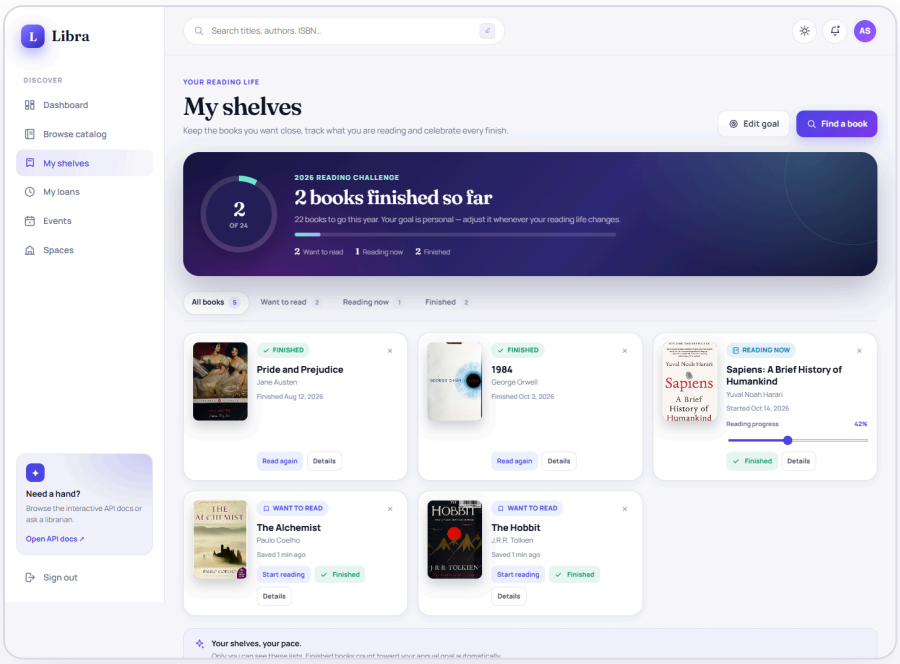
What is new?

Recommendations are grounded in rating and availability, then linked to the same catalog record.

> "The next book should already feel close at hand."
>
> The member experience principle behind the home screen

The books you mean to read.

Discovery is not finished when a book is borrowed. Libra gives readers a private place to save what catches their eye, track what they are reading and finish the year with a record of everything they completed.



My shelves. A goal ring, shelf filters and counts, then a working card for every saved title. Progress is editable in place; finishing moves the book into the annual goal immediately.

Three states, one source of truth

- **Want to read** — saved from any catalog card with the bookmark pin.
- **Reading now** — a progress slider and a started date keep the current book moving.
- **Finished** — a finish timestamp feeds the yearly goal; “Read again” reopens the book.

ANNUAL GOAL

Members choose a target of 1–365 books. The dashboard, profile and shelves page all read the same calculated progress. A 24-book goal with two finishes shows 2 of 24 — never a separate counter that can drift.

PRIVATE BY DEFAULT

Shelves belong to the signed-in member. Staff cannot read or edit another member’s shelves; the API enforces ownership on every read and write.

3
shelves: want to read, reading now and finished

1
progress slider feeding a calculated yearly goal

0
denormalized goal counters that can drift out of sync

RESEARCH SIGNAL

Libby, Literal and Aspen Discovery all make saved, in-progress and finished reading visible. Libra adopts that familiar three-shelf model and adds a library-native yearly goal rather than a social feed.

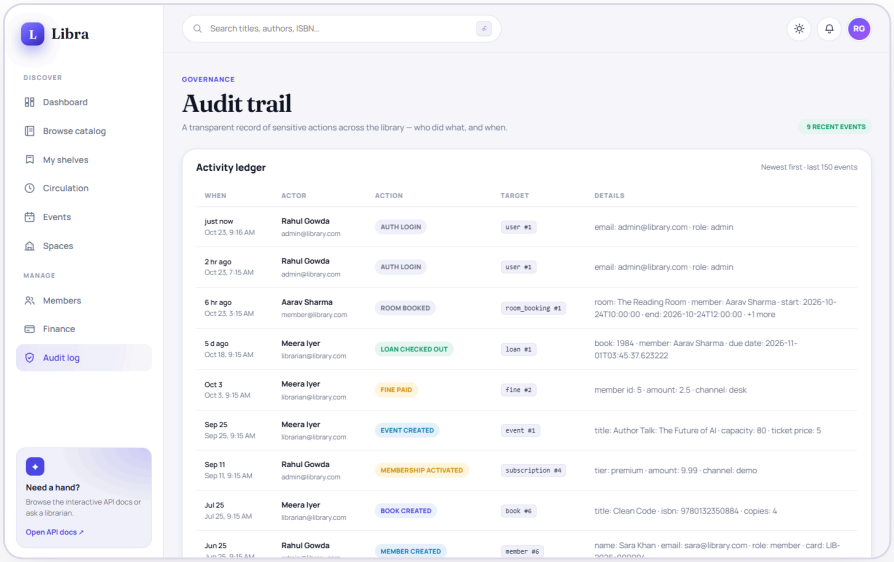
[19][20][21]

MOTION WITH RESTRAINT

The staggered login, password reveal, book-cover transition and closing-book logout are short and purposeful. [22] Every one has a reduced-motion alternative, so the interface never withholds content behind decoration.

> “A saved book is a small promise. The system should remember it.”
>
> Why shelves live beside discovery, not inside a separate account screen

The record is part of the product.



Audit ledger, staff-only. Newest first, filterable by action and entity, with actor, target, structured details and timestamp.

Security posture

- JWT access and refresh tokens; bcrypt with PBKDF2 fallback.
- Role guards on staff-only endpoints and UI routes.
- Passwords never returned; session tokens stored client-side for the demo.
- Audit writes share the business transaction; failures roll back together.
- Stripe webhook signatures supported; secrets stay server-side.
- No real credentials or keys committed to the repository.

Audited

Auth, member, catalog, circulation, fine, payment, event and room actions.

Guarded

Members cannot read the ledger, change roles or inspect another member's fines and payments.

Operable

Health endpoint, API docs, typed schemas and deterministic local seed make recovery and demos repeatable.

> Trust is a feature: if a librarian cannot see who changed a record, the system has not finished its job.

What gets logged

ACTION	TARGET	WHAT THE ROW CAPTURES
CHECKOUT	Loan + copy	Actor, member, book, due date, branch and timestamp.
RETURN	Loan + copy	Return condition, days late, fine created and resulting copy state.
FINE_PAID	Fine + payment	Amount, method, Stripe session or development-mode settlement.
MEMBER_CREATED	User	Role, tier, card number and the staff member who created the account.
ROOM_BOOKED	Room booking	Room, member, start/end time and the overlap check that allowed it.
SHELF_UPDATED	ReadingShelf	Member, book, shelf state and progress; goal changes are logged as `reading_goal.updated`.
LOGIN	Session	Successful sign-in, channel and the role attached to the token.

Operational privacy: the ledger is staff-only, paginated and filterable; member-facing views never expose other readers' activity. Rows store structured identifiers and action details — never passwords, tokens or card numbers.

Simple on purpose.

A single FastAPI process serves the API and the SPA. SQLite makes the demo instant; PostgreSQL is a DSN change. There is no Node build step and no hidden service to operate.

Browser

Vanilla-JS SPA · accessible shell · responsive CSS · PWA service worker

HTML / CSS / JS

HTTPS JSON →

FastAPI

12 route modules · Pydantic v2 schemas · JWT dependencies · role guards · audit helpers

Python 3.11+

SQLAlchemy 2.0 →

Database

SQLite by default; PostgreSQL in Docker or production. Typed models, enums, timestamps and relationships.

SQLite / Postgres

Request lifecycle

1. JWT dependency resolves the current user and role.
2. Pydantic validates the request and serialises the response.
3. The route performs the business action inside a DB session.
4. An audit row is added in the same transaction.
5. Commit returns fresh data; errors roll back cleanly.

Data model highlights

User · **ReadingShelf** · **Book / BookCopy** · **Author / Publisher / Category / Branch** · **Loan / Reservation / Fine** · **Payment / MembershipSubscription** · **Event / EventRegistration** · **Room / RoomBooking** · **Review / Notification / AuditLog**.

Branch-aware copies and events leave room for scoped staffing and hard multi-tenancy; the current build is one deployment per tenant.

WHY THIS SHAPE

Small teams can run it with one Python process and one database file. Production can move to PostgreSQL and a reverse proxy without rewriting the application. The boundaries are visible in the code, not hidden in framework magic.

Requests, in practice

```
# checkout a copy for a member
POST /api/loans
{ "book_copy_id": 12, "user_id": 4 }

# settle a fine through hosted Checkout
POST /api/payments/fines/checkout
{ "fine_ids": [1, 2] }
```

Project shape

```
backend/
  api/routes/      12 modules
  models/          SQLAlchemy 2
  schemas/         Pydantic v2
  auth/            JWT + bcrypt
  database/        seed + session
web/
  app.js  styles.css  service-worker.js
```

Health first

GET /api/health returns service status, version and payment mode.

Docs included

OpenAPI at **/api/docs** exposes every endpoint and schema for inspection.

Repeatable demos

A deterministic seed makes every screenshot and test run start from the same library.

Checkout that gets out of the way.

With Stripe keys, Libra redirects to hosted Stripe Checkout for fines, event tickets and memberships. Without keys, every flow still works in a clearly labelled development mode.

01 · Choose
Member selects fines or a tier; staff register a paid event ticket.

→

02 · Checkout
Server creates a Checkout Session with metadata and returns the hosted URL.

→

03 · Return
The SPA confirms the session on return, or a signed webhook settles it server-side.

→

04 · Settle
Payment, fine, membership or registration state updates once; an audit row records it.

What Stripe handles

- Card data and PCI scope stay on Stripe's hosted page.
- Wallets such as Apple Pay and Google Pay appear when enabled on the account.
- Subscription billing and first-invoice amounts.
- Signed `checkout.session.completed`, `checkout.session.expired`, `payment_intent.succeeded` and `invoice.paid` events.

What Libra owns

- Which fines are unpaid and which member owns them.
- Server-computed amounts, not client-supplied prices.
- Idempotent settlement helpers so a webhook retry cannot double-pay a fine.
- Membership tier activation and abandoned-checkout cleanup.

FLOW	ENTRY POINT	COMPLETION
Fine payment	**POST /payments/fines/checkout**	Hosted Checkout return confirmation or webhook; fine becomes paid.
Membership	**POST /payments/subscribe**	Subscription Checkout; tier and 30-day period activated; first payment recorded.
Event ticket	**POST /events/{id}/register**	Ticket Checkout; registration held, then settled; expired sessions release the seat.
Development	Same endpoints, no keys	Instant settlement with an explicit "development mode" message.

Payment states

pending	Checkout created; no entitlement yet.
paid	Fine closed, membership active or seat confirmed.
failed	No entitlement; member can retry safely.
expired	Abandoned session; held event seat released.

Failure handling

- Settlement is idempotent: a repeated webhook or return confirmation cannot double-charge a fine.
- Server-side amounts are recomputed from the database, never trusted from the browser.
- An expired hosted session releases a held event seat rather than blocking the room forever.
- If keys are removed, the same endpoints fall back to development mode with a visible label.

PCI scope: card data is entered on Stripe's hosted page. Libra stores payment status and identifiers, not card numbers.

FINES
Select one or all unpaid fines; the server totals the amount and closes only what was settled.

MEMBERSHIPS
Basic, Student, Premium and Family tiers; the first invoice is recorded and the period starts on payment.

TICKETS
Free and paid events share the same capacity and registration model; paid seats are held during Checkout.

From clone to working library.

A fresh SQLite database seeds a realistic library on first start. The demo accounts make every role reachable in seconds; the test suite keeps the critical paths honest.

Start

Windows: `.\start.ps1`
Creates the venv, installs dependencies, copies safe defaults, seeds and runs Uvicorn.

Any OS: `uvicorn api.main:app --port 8000`
App at **http://localhost:8000** · API docs at **/api/docs**.

Configure

DATABASE_URL	SQLite default; PostgreSQL DSN supported.
APP_BASE_URL	Browser-visible URL for Stripe return links.
STRIPE_SECRET_KEY	Enables hosted Checkout.
STRIPE_WEBHOOK_SECRET	Verifies production webhooks.
SECRET_KEY	Change before any real deployment.

Demo accounts

Administrator	admin@library.com Admin@123
Librarian	librarian@library.com Librarian@123
Member	member@library.com Member@123

Additional readers: priya@, john@ and sara@library.com (Member@123).

Verification

PASS API smoke test — auth, catalog, circulation, reviews, shelves, goals, fines, memberships, events, rooms, access control, PWA.

PASS Browser UI test — desktop and 390 px mobile, shelves, goals, light/dark, PWA, auth animations.

PASS Offline Stripe test — hosted fine Checkout, return confirmation and idempotent settlement.

PASS Screenshot capture — 20 reviewed states from a pristine seed.

DEPLOYMENT

`docker compose up --build` starts PostgreSQL and the API. The web app is served by the same container. Use a strong secret, a public base URL, HTTPS, migrations, backups and rate limiting before production.

RESET THE DEMO

Delete `backend/library.db` and restart. The deterministic seeder recreates the library, accounts, loans, events, rooms and audit history on first boot.

No production data lives in the demo database; the final screenshots were captured from a pristine seed.

FIRST-RUN CHECKLIST

- ✓ Open **http://localhost:8000** and sign in with a demo role.
- ✓ Check **/api/health** and **/api/docs**.
- ✓ Follow one book through checkout, return and review.
- ✓ Run the smoke, UI and Stripe test scripts.

Honest edges.

Libra is a complete, runnable local product and a credible foundation. It is not a hardened multi-tenant SaaS. These boundaries are deliberate and visible.

Deliberate boundaries

- **Email/SMS:** configuration exists; no external delivery provider is wired. In-app notifications are fully functional.
- **Digital content:** links to public-domain Project Gutenberg editions; DRM-protected e-lending and loan expiry are out of scope.
- **AI:** catalog suggestions and recommendations are transparent local heuristics, not hosted generative models.
- **Multi-tenant:** branches and per-copy assignment are modelled; hard tenant isolation and branch-scoped staff permissions are a next step. Run one deployment per tenant.
- **Internationalisation:** UTC timestamps, configurable currency and international fields; full translated catalogues and RTL layouts are not included.

Production checklist

- PostgreSQL + Alembic migrations, not create-all-on-start.
- Strong `SECRET_KEY`, HTTPS, secure cookies, rate limiting.
- Signed Stripe webhooks and a public `APP_BASE_URL`.
- Backups, retention policy and audit-log shipping.
- Branch-scoped permissions, accessibility audit, localisation.

> A product is more trustworthy when it can say what it does not do.

THE INVITATION

Run it with the demo seed, open two browsers — one staff, one member — and follow a book from the shelf to a due date, a fine, a payment and the audit ledger. The whole loop is there.

What comes next

Now

Catalog, circulation, members, personal shelves and goals, fines, payments, events, rooms, analytics, audit, PWA, animated auth and responsive UI.

Next 90 days

MARC/BIBFRAME import-export, email templates, branch transfers, reservation pickup, donation and acquisition flows.

Later

Hard tenant isolation, DRM/e-lending, full translation and RTL, native wrappers, advanced accessibility audit.

DECISION	WHY IT WAS CHOSEN
SQLite by default	Zero-install evaluation; PostgreSQL remains a DSN change for production.
Hosted Checkout	Keeps card data out of the app and supports wallets and local payment methods.
Transactional audit	A record that cannot be committed separately from the action is more trustworthy.

Sources & further reading.

The product direction was informed by the following sector reports, ILS practice, platform standards and public documentation. Links are included for verification.

1. **American Library Association.** *State of America's Libraries Report 2026.* ala.org/news/state-americas-libraries-report-2026
2. **International Federation of Library Associations.** *IFLA Trend Report 2024.* ifla.org/trend-report
3. **IFLA / UNESCO.** *Public Library Manifesto.* ifla.org/public-library-manifesto
4. **Institute of Museum and Library Services.** *Public Libraries Survey (PLS).* imls.gov/.../public-libraries-survey-pls
5. **Association of College & Research Libraries.** *State of Academic Libraries Report 2024.* ala.org/.../ACRL_State_of_Academic_Libraries_Report 2024
6. **Koha Community.** *Koha Documentation.* koha-community.org/documentation
7. **Library of Congress.** *BIBFRAME.* loc.gov/bibframe
8. **Boston Public Library.** *Reservable Community Spaces.* bpl.org/reservable-community-spaces
9. **King County Library System.** *Meeting & Study Rooms.* kcls.org/rooms
10. **DC Public Library.** *Meeting and Study Room Guide.* dclibrary.org/reserve-room/meeting-and-study-room-guide
11. **Stripe.** *Stripe Checkout.* docs.stripe.com/payments/checkout
12. **Stripe.** *Payment Intents API.* docs.stripe.com/payments/payment-intents
13. **SQLAlchemy.** *SQLAlchemy 2.0 Documentation.* docs.sqlalchemy.org/en/20
14. **FastAPI.** *FastAPI Documentation.* fastapi.tiangolo.com
15. **W3C.** *Web Application Manifest.* w3.org/TR/appmanifest
16. **W3C.** *Web Content Accessibility Guidelines (WCAG) 2.2.* w3.org/TR/WCAG22
17. **American Library Association / Public Library Association.** *Public Library Services Report 2026.* ala.org/.../PLA_Services_Report_2026.pdf
18. **Seattle Public Library.** *New mobile app for patrons, 2026.* spl.org/.../new-mobile-app-for-patrons
19. **OverDrive.** *Libby reading app.* play.google.com/store/apps/details?id=com.overdrive.mobile.android.libby
20. **Literal.** *Discover books — reading tracker and yearly goal.* apps.apple.com/us/app/literal-discover-books/id1566119883
21. **ByWater Solutions.** *Reading History & Privacy in Aspen Discovery and Koha.* bywatersolutions.com/education/patron-reading-history-in-aspen-discovery-and-koha
22. **Nielsen Norman Group.** *Animation duration and perceived performance.* nngroup.com/articles/animation-duration
23. **OWASP.** *Logging Cheat Sheet.* cheatsheetseries.owasp.org/.../Logging_Cheat_Sheet.html
24. **NIST.** *SP 800-92: Guide to Computer Security Log Management.* csrc.nist.gov/pubs/sp/800/92/final
25. **Open Library.** *Covers API.* openlibrary.org/dev/docs/api/covers
26. **Project Gutenberg.** *Free public-domain eBooks.* gutenberg.org
27. **Unicode Consortium.** *CLDR.* unicode.org/cldr
28. **ISO.** *ISO 4217 Currency Codes.* iso.org/iso-4217-currency-codes.html
29. **PostgreSQL.** *Row Security Policies.* postgresql.org/docs/current/ddl-rowsecurity.html

The bibliography was checked during the build. Libra's demo data is fictional and for demonstration; book metadata and covers are attributed to their respective sources. Digital edition links point to Project Gutenberg. This document describes the implementation in this workspace, including its explicit boundaries.

How to verify this overview

OPEN THE APP

http://localhost:8000
Administrator: admin@library.com / Admin@123
Member: member@library.com / Member@123

FOLLOW ONE LOOP

Check out a copy, let it become overdue, return it, settle the fine, then open the audit ledger as staff.

READ THE EVIDENCE

Screens in **docs/screenshots**; tests in **backend/tests**; implementation notes in **README.md**.

Libra — Library OS · product overview · v1.0.0
Prepared October 2026 · 15 pages · 20 documented screens

Live demo at **http://localhost:8000**
No production keys or personal data are included.